Project Loom

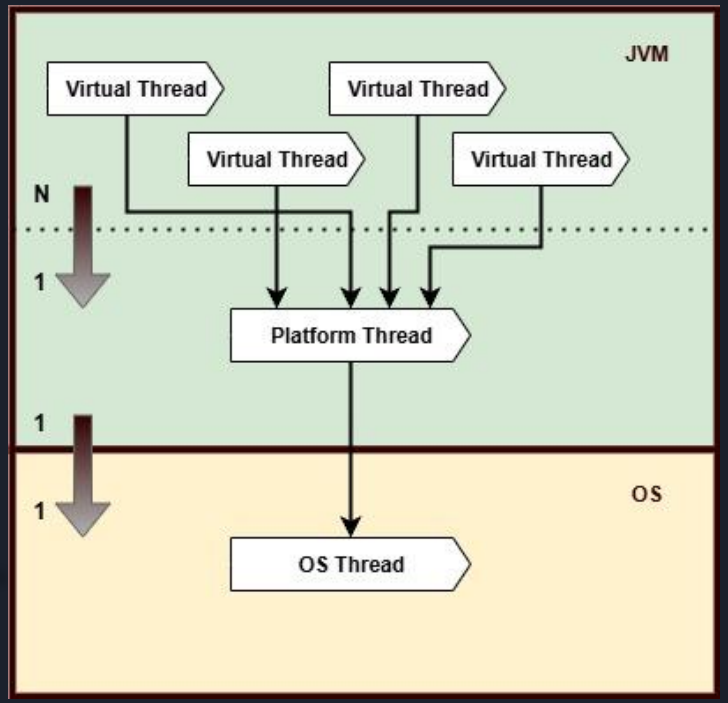
Author: Konrad Kowalczyk
Repository: xkondix loom

Presentation Plan

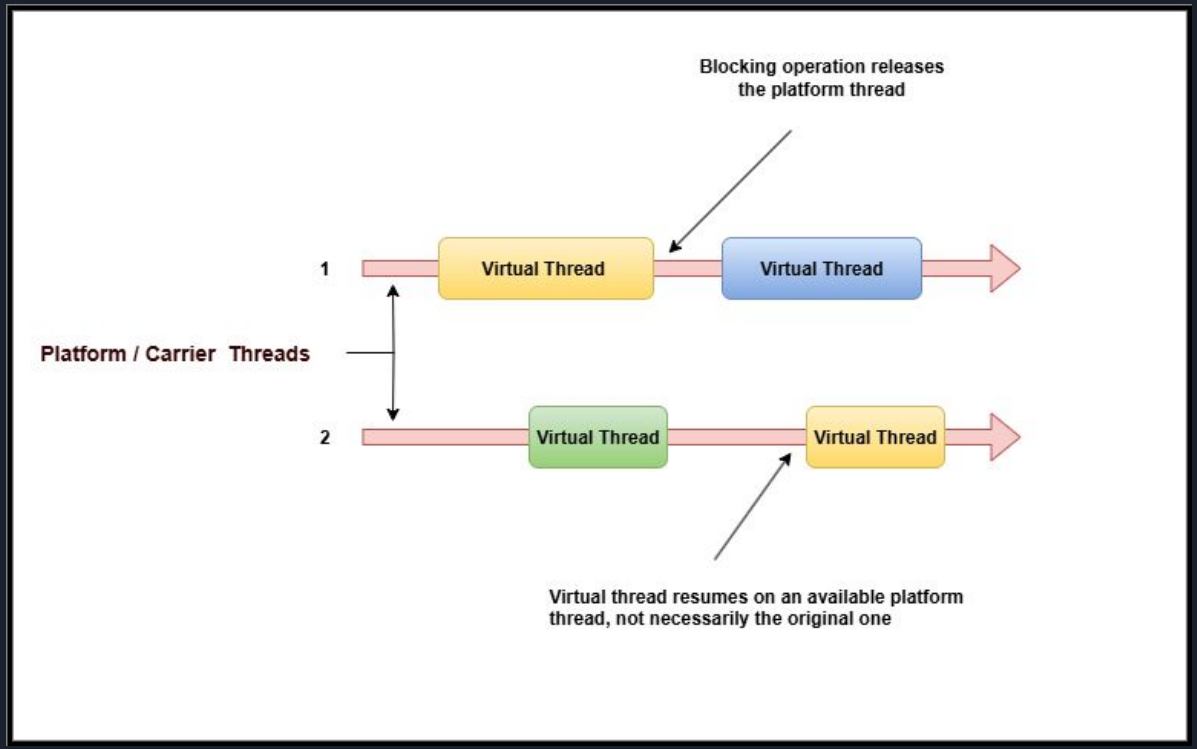
- Virtual Threads
- Continuation
- Virtual Threads Examples
- Scoped Value
- Structured Concurrency
- Virtual Threads + Spring Boot
- Virtual Threads vs Completable Future
- Questions

Virtual & Platform & OS Threads

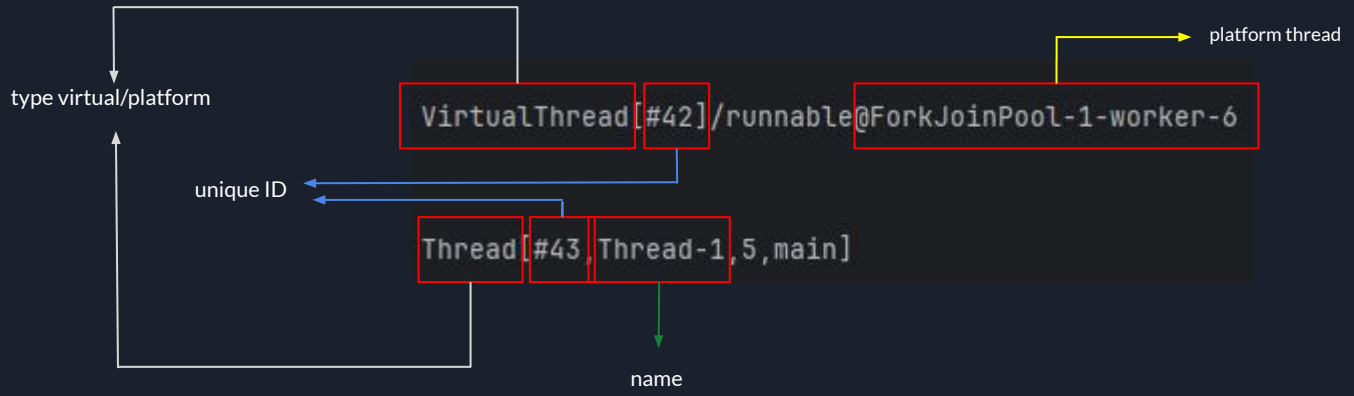
- Both **Platform** and **Virtual Threads** are **managed by the JVM**
- **Platform Threads** are heavyweight and have a **1:1** mapping with OS threads
- **Virtual Threads** are lightweight and have an **n:1** mapping to Platform Threads
- **Virtual Threads** are ideal for tasks with blocking I/O or high concurrency needs
- JVM's **scheduler** handles Virtual Thread execution by reusing Platform Threads
- Blocking a Virtual Thread does **not** block the OS Thread thanks to continuation-based scheduling



Virtual Thread Scheduler



Thread.currentThread()



- A virtual thread is assigned to a platform thread
- While it is assigned, the platform thread is the carrier (or carrier thread) for the virtual thread

Threads managed by the JVM

- **Platform Thread**
  A traditional Java thread that wraps a native **Operating System (OS) thread**.
  It is **created and managed by the JVM**, but its **execution is scheduled by the OS thread scheduler**.

- **Virtual Thread**
  A **lightweight thread** managed by the **JVM's virtual thread scheduler**.
  It can be **mounted** (bound) to a platform thread for execution when running code.
  Virtual threads are ideal for highly concurrent, non-blocking tasks.

- **Carrier Thread**
  A **platform thread** that temporarily **executes a virtual thread's code** during its active phase.
  While mounted, the platform thread acts as the **carrier** for the virtual thread.

Continuation

⚠️ **Continuation** is an internal JDK class that encapsulates code which can be paused and resumed later, and is not intended for public use.

File

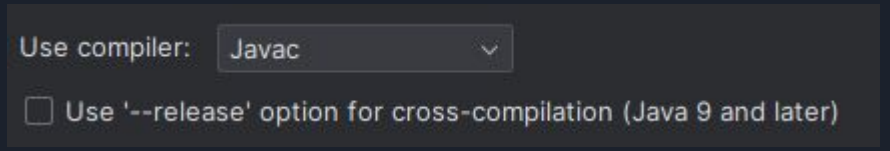
→ Settings (or Ctrl+Alt+S)

→ Build, Execution, Deployment

→ Compiler

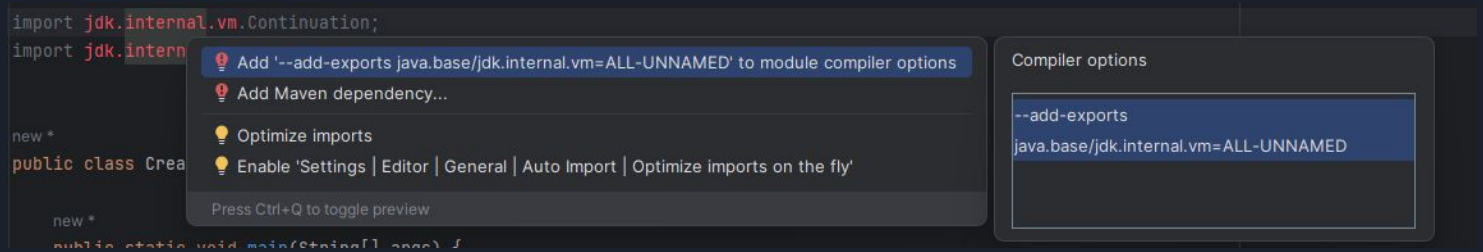
→ Java Compiler

You need uncheck the **release** box



Unlock internal classes IDEA

Click CTRL + ENTER -> ADD `import jdk.internal.vm.Continuation;`




```
<component name="JavacSettings">
  <option name="ADDITIONAL_OPTIONS_OVERRIDE">
    <module name="loom" options="--add-exports java.base/jdk.internal.vm=ALL-UNNAMED" />
  </option>
</component>
```

Unlock internal classes Java Compiler

File

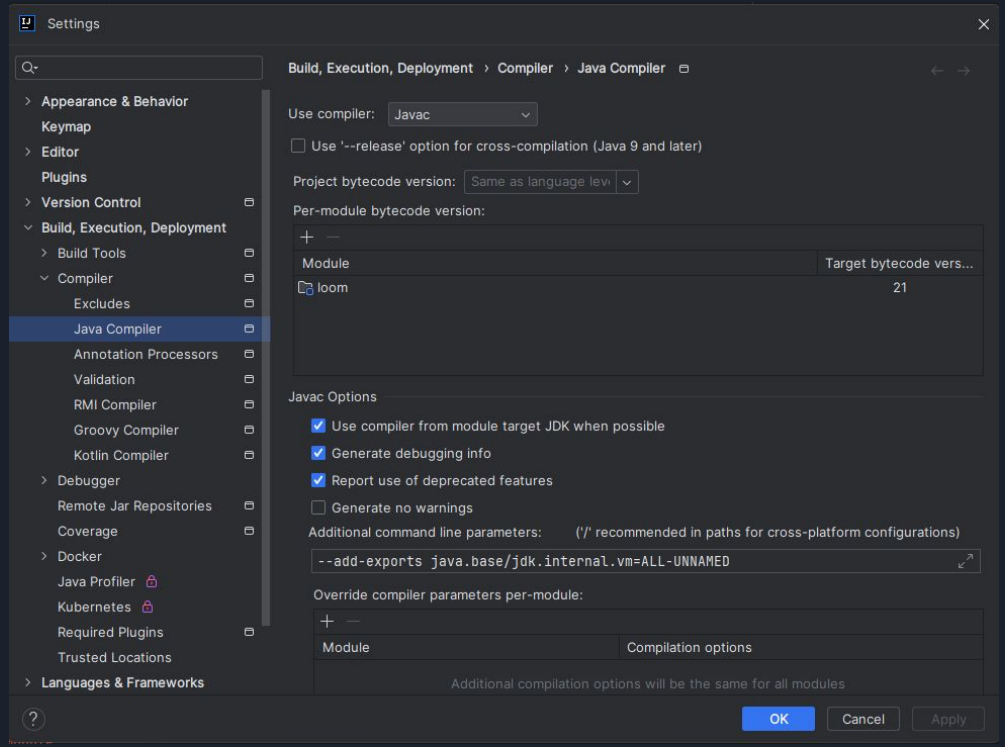
→ Settings (or Ctrl+Alt+S)

→ Build, Execution, Deployment

→ Compiler

→ Java Compiler

```
--enable-preview --add-exports
java.base/jdk.internal.vm=ALL-UNNAMED
```



Unlock internal classes VM Options

Steps:

Run

→ **Edit Configurations** (from the top menu)

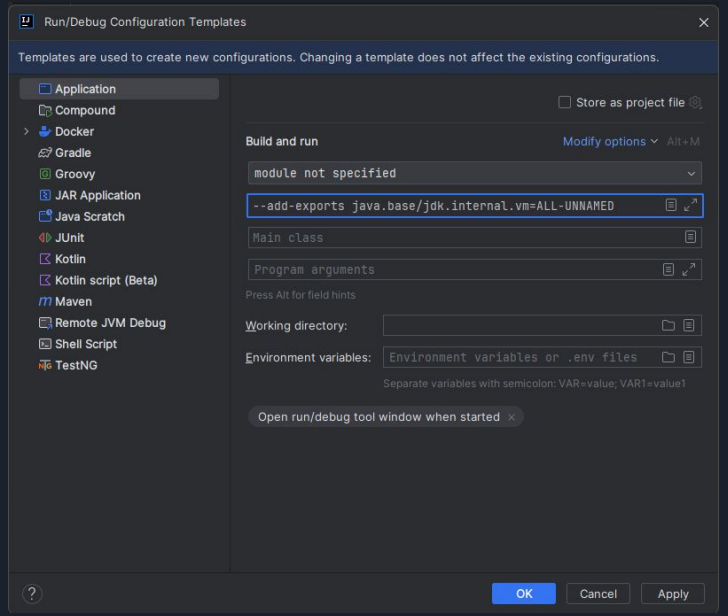
→ in the bottom-left corner, click **Edit configuration templates**

→ select **Application**

→ in the **VM options** field, enter:

```
--enable-preview --add-exports java.base/jdk.internal.vm=ALL-UNNAMED
```

Click **Apply**, then **OK**.

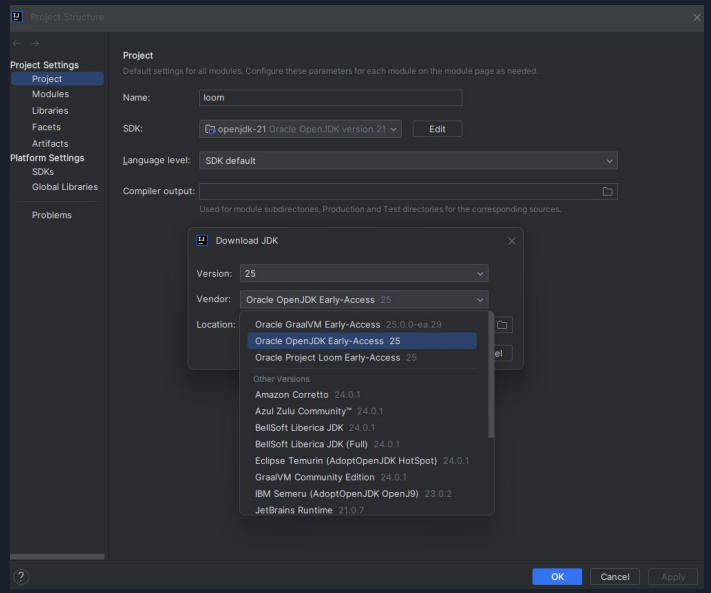


JAVA 25

Option 1

https://jdk.java.net./25/

Option 2



Virtual Thread Scheduler

VM options `-Djdk.virtualThreadScheduler.parallelism=2`

```
Task 2 start on - VirtualThread[#24]/runnable@ForkJoinPool-1-worker-2
Task 2 sleep 3s
Task 1 start on - VirtualThread[#22]/runnable@ForkJoinPool-1-worker-1
Task 1 sleep 2s
Task 3 start on - VirtualThread[#25]/runnable@ForkJoinPool-1-worker-1
Task 3 end on - VirtualThread[#25]/runnable@ForkJoinPool-1-worker-2
Task 1 restart on - VirtualThread[#22]/runnable@ForkJoinPool-1-worker-2
Task 2 restart on - VirtualThread[#24]/runnable@ForkJoinPool-1-worker-1
```

Pinning

Pinning occurs when the virtual thread does not release the platform thread.

JAVA 21

- synchronized blocks ❌
- datagram sockets ❌
- DNS lookups ❌ ?
- native blocking code ❌
- file operations ❌ ?

JAVA 25

- synchronized blocks ✅
- datagram sockets ❌ ?
- DNS lookups ❌ ?
- native blocking code ❌
- file operations ❌ ?

Synchronized blocks

vm options

```
-Djdk.tracePinnedThreads=short -Djdk.virtualThreadScheduler.parallelism=1
```

JAVA 25

```
Start: VirtualThread[#28]/runnable@ForkJoinPool-1-worker-1
End: VirtualThread[#28]/runnable@ForkJoinPool-1-worker-1
Start: VirtualThread[#30]/runnable@ForkJoinPool-1-worker-1
End: VirtualThread[#30]/runnable@ForkJoinPool-1-worker-2
```

JAVA 21

```
Start: VirtualThread[#22]/runnable@ForkJoinPool-1-worker-1
Thread[#23,ForkJoinPool-1-worker-1,5,CarrierThreads]
    com.kowalczyk.konrad.loom.virtual.pinning.PinningSynchronized.lambda$main$0(PinningSynchronized.java:15) <== monitors:1
End: VirtualThread[#22]/runnable@ForkJoinPool-1-worker-1
Start: VirtualThread[#24]/runnable@ForkJoinPool-1-worker-2
End: VirtualThread[#24]/runnable@ForkJoinPool-1-worker-2
```

ReentrantLock

vm options

```
-Djdk.tracePinnedThreads=short -Djdk.virtualThreadScheduler.parallelism=1
```

JAVA 25

```
Start: VirtualThread[#28]/runnable@ForkJoinPool-1-worker-1
End: VirtualThread[#28]/runnable@ForkJoinPool-1-worker-1
Start: VirtualThread[#30]/runnable@ForkJoinPool-1-worker-1
End: VirtualThread[#30]/runnable@ForkJoinPool-1-worker-2
```

JAVA 21

```
Start: VirtualThread[#22]/runnable@ForkJoinPool-1-worker-1
Thread[#23,ForkJoinPool-1-worker-1,5,CarrierThreads]
    com.kowalczyk.konrad.loom.virtual.pinning.PinningReentrantLock.lambda$main$0(PinningReentrantLock.java:15) <== monitors:1
End: VirtualThread[#22]/runnable@ForkJoinPool-1-worker-1
Start: VirtualThread[#24]/runnable@ForkJoinPool-1-worker-2
End: VirtualThread[#24]/runnable@ForkJoinPool-1-worker-2
```

IO operations

I have not been able to reproduce it, but based on the documentation it is probably possible in both JAVA 21 and JAVA 25.

Nevertheless, many operations are safe in both versions. More in the documentation.

VM options `-Djdk.tracePinnedThreads=full`

link to doc -> https://openjdk.org/jeps/444#java-io

Datagram Socket

VM options

`-Djdk.tracePinnedThreads=short`

JAVA 25

```
Waiting to receive UDP packet... VirtualThread[#28]/runnable@ForkJoinPool-1-worker-1
Received onVirtualThread[#28]/runnable@ForkJoinPool-1-worker-1 data - Hello from client

Process finished with exit code 0
```

JAVA 21

```
Waiting to receive UDP packet... VirtualThread[#22]/runnable@ForkJoinPool-1-worker-1
Thread[#23,ForkJoinPool-1-worker-1,5,CarrierThreads]
    java.base/sun.nio.ch.DatagramSocketAdaptor.receive(DatagramSocketAdaptor.java:241) <== monitors:1
Received onVirtualThread[#22]/runnable@ForkJoinPool-1-worker-1 data - Hello from client
```

DNS lookups

I have not been able to reproduce it, but based on the documentation it is probably possible in both JAVA 21 and JAVA 25.

VM options `-Djdk.tracePinnedThreads=full`

link to doc -> https://openjdk.org/jeps/444#Networking

Native blocking code

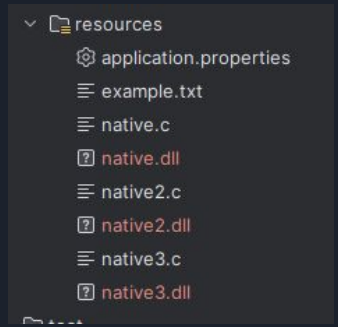
“There are two scenarios in which a virtual thread cannot be unmounted during blocking operations because it is pinned to its carrier:

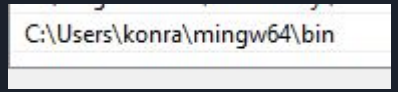
- When it executes code inside a synchronized block or method
- When it executes a native method or a foreign function “

1. download gcc https://winlibs.com/#download-release
2. add to path bin folder
3. cd .\src\main\resources
4. run:

- gcc -I"$env:JAVA_HOME\include" -I"$env:JAVA_HOME\include\win32" -shared -o native.dll native.c
- gcc -I"$env:JAVA_HOME\include" -I"$env:JAVA_HOME\include\win32" -shared -o native2.dll native2.c
- gcc -I"$env:JAVA_HOME\include" -I"$env:JAVA_HOME\include\win32" -shared -o native3.dll native3.c

5. VM options **-Djdk.tracePinnedThreads=full -Djava.library.path=C:\Users\konra\Desktop\loom\src\main\resources**





Scoped Value

A **ScopedValue** is a new way in Java to safely share *immutable data* across methods and threads, **without using thread-local variables**.

It allows you to:

- Bind a value to a limited dynamic scope
- Pass context (like user info, config, etc.) to other methods or threads in a safe and clean way
- Avoid common problems of ThreadLocal, such as leaks or incorrect inheritance

Scoped values are **immutable**, **inheritable**, and **automatically cleared** when out of scope.

JAVA 21
(Preview)

https://openjdk.org/jeps/446

JAVA 25
(Release)

https://openjdk.org/jeps/506

ThreadLocal vs ScopedValue

Feature	ThreadLocal	ScopedValue
Mutability	Mutable – the value can be set or changed at any time.	Immutable reference – value is fixed for the duration of the scope (object itself can still be mutable).
Scope	Bound to the entire thread lifetime (manual cleanup required).	Bound to a well-defined dynamic scope (ScopedValue.where(...).run(...) or .call(...)).
Thread Safety	Easy to misuse – may leak values or accidentally share between threads.	Safe by design – no accidental sharing or leaks.
Virtual Threads	Not recommended – can cause memory leaks or unexpected behavior due to carrier thread reuse.	Designed to work well with virtual threads and structured concurrency.
Inheritance	Supports implicit inheritance (InheritableThreadLocal), but this is often error-prone.	No implicit inheritance – all context passing is explicit.
Cleanup	Must be manually cleared to avoid leaks.	Automatically cleared when the scope exits.
Performance	Can be slower and more error-prone in modern concurrent environments; fine for simple thread-bound values.	Often faster in concurrent/virtual thread scenarios and reduces logic errors.

Structured Concurrency

- **StructuredTaskScope.ShutdownOnFailure()** – cancels the remaining tasks on the first failure
- **StructuredTaskScope.ShutdownOnSuccess()** – stops the others as soon as one succeeds (e.g., for racing)

JAVA 21
(Preview)

https://openjdk.org/jeps/453

JAVA 25
(Fifth Preview)

https://openjdk.org/jeps/505

Structured Concurrency + Scoped Value

```
findName running in: VirtualThread[#22]/runnable@ForkJoinPool-1-worker-1
StructuredTaskScope running in: Thread[#1,main,5,main]
findName - User: Konrad
findID running in: VirtualThread[#24]/runnable@ForkJoinPool-1-worker-2
findID - User: Konrad
after sleep findID running in: VirtualThread[#24]/runnable@ForkJoinPool-1-worker-3
Final result for Konrad:
- Name: Konrad
- ID: 98
- ID form scope: id_test
userWithoutScoped - ScopedValue not set: java.util.NoSuchElementException

Process finished with exit code 0
```

Virtual Threads + Spring Boot

To use Virtual Thread with spring boot you need add to properties `spring.threads.virtual.enabled=true`

This is not always true

An example of virtual threads used by Spring Boot is shown below (code on the next slide)

```
2025-08-09T22:18:57.590+02:00  INFO 8036 --- [loom] [omcat-handler-3] c.k.konrad.loom.spring.UserService       : Thread before db - VirtualThread[#67,tomcat-handler-3]/runnable@ForkJoinPool-1-worker-2
Hibernate: select u1_0.id,c1_0.id,c1_0.latitude,c1_0.longitude,c1_0.name,u1_0.name from user u1_0 join city c1_0 on c1_0.id=u1_0.city_id where u1_0.name=?
2025-08-09T22:18:57.599+02:00  INFO 8036 --- [loom] [omcat-handler-3] c.k.konrad.loom.spring.UserService       : Thread before client and after db - VirtualThread[#67,tomcat-handler-3]/runnable@ForkJoinPool-1-worker-2
2025-08-09T22:18:59.651+02:00  INFO 8036 --- [loom] [omcat-handler-3] c.k.konrad.loom.spring.UserService       : Thread  after client - VirtualThread[#67,tomcat-handler-3]/runnable@ForkJoinPool-1-worker-3
2025-08-09T22:18:59.652+02:00  INFO 8036 --- [loom] [omcat-handler-3] c.k.konrad.loom.spring.UserService       : Time of execution: 2.062 ms
```


```
  : Thread before db - VirtualThread[#67,tomcat-handler-3]/runnable@ForkJoinPool-1-worker-2
1_0 on c1_0.id=u1_0.city_id where u1_0.name=?
  : Thread before client and after db - VirtualThread[#67,tomcat-handler-3]/runnable@ForkJoinPool-1-worker-2
  : Thread  after client - VirtualThread[#67,tomcat-handler-3]/runnable@ForkJoinPool-1-worker-3
  : Time of execution: 2.062 ms
```

Virtual Threads + Spring Boot

```
1 usage  new *
public UserWeatherPojo getUserWithWeatherVirtual(String name) {
    long start = System.currentTimeMillis();
    log.info("Thread before db - {}", Thread.currentThread());

    User user = userRepository.findByNameWithCity(name)
            .orElseThrow(() -> new RuntimeException("User not found"));

    log.info("Thread before client and after db - {}", Thread.currentThread());
    try {
        Thread.sleep( millis: 2000);
    } catch (InterruptedException e) {
        throw new RuntimeException(e);
    }
    String temp = client.getTemperatureSync(
            user.getCity().getLatitude(),
            user.getCity().getLongitude());

    log.info("Thread  after client - {}", Thread.currentThread());
    log.info("Time of execution: {} ms", (System.currentTimeMillis() - start) / 1000.0);

    return new UserWeatherPojo(user.getName(), user.getCity().getName(), temp);
}
```

Code comparison (Completable Future)

```
public CompletableFuture<UserWeatherPojo> getUserWithWeather(String name) {
    return CompletableFuture.supplyAsync(() ->
            userRepository.findByNameWithCity(name)
                    .orElseThrow(() -> new RuntimeException("User not found"))
    ).thenCompose(user ->
            client.getTemperatureAsync(
                    user.getCity().getLatitude(),
                    user.getCity().getLongitude()
            ).thenApply(temp ->
                    new UserWeatherPojo(user.getName(), user.getCity().getName(), temp)
            )
    );
}
```

Code comparison (Virtual Threads)

```
public UserWeatherPojo getUserWithWeatherVirtual(String name) {

    User user = userRepository.findByNameWithCity(name)
            .orElseThrow(() -> new RuntimeException("User not found"));

    String temp = client.getTemperatureSync(
            user.getCity().getLatitude(),
            user.getCity().getLongitude());

    return new UserWeatherPojo(user.getName(), user.getCity().getName(), temp);
}
```

Completable Future

Results between 51 ms and 223 ms


```
Thread before DB query - Thread[#60,ForkJoinPool.commonPool-worker-1,5,main]
on c1_0.id=u1_0.city_id where u1_0.name=?
Thread after DB query - Thread[#60,ForkJoinPool.commonPool-worker-1,5,main]
Thread before calling weather API - Thread[#60,ForkJoinPool.commonPool-worker-1,5,main]
Thread after receiving weather API response - Thread[#60,ForkJoinPool.commonPool-worker-1,5,main]
Total time: 54 ms
```

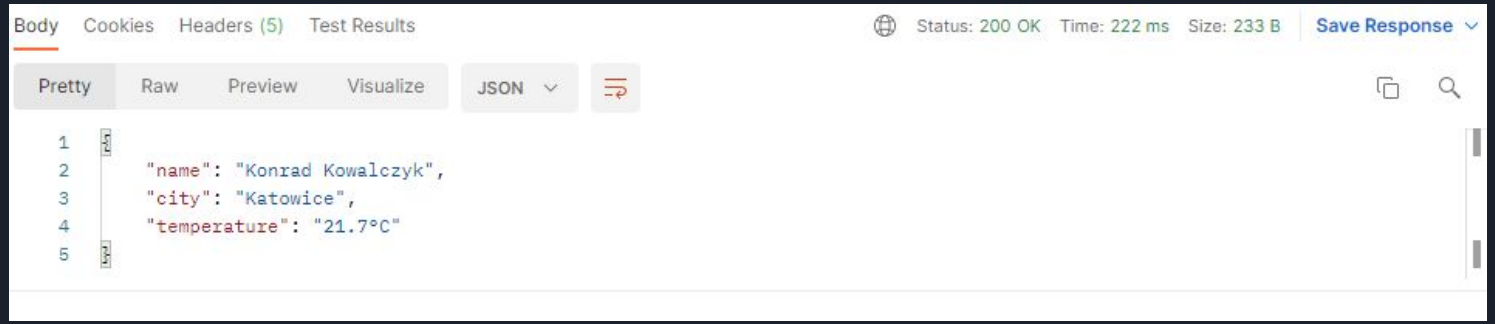
Blocking code

Results between 50 ms and 230 ms


```
 : Thread before DB query - Thread[#48,http-nio-8080-exec-2,5,main]
0 on c1_0.id=u1_0.city_id where u1_0.name=?
 : Thread before calling weather API and after DB query - Thread[#48,http-nio-8080-exec-2,5,main]
 : Thread after receiving weather API response - Thread[#48,http-nio-8080-exec-2,5,main]
 : Time of execution: 0.063 ms
```

Completable Future + Virtual Threads

Results between 50 ms and 220 ms




```
 : Thread before DB query - Thread[#72,ForkJoinPool.commonPool-worker-2,5,VirtualThreads]
.0 on c1_0.id=u1_0.city_id where u1_0.name=?
 : Thread after DB query - Thread[#72,ForkJoinPool.commonPool-worker-2,5,VirtualThreads]
 : Thread before calling weather API - Thread[#72,ForkJoinPool.commonPool-worker-2,5,VirtualThreads]
 : Thread after receiving weather API response - Thread[#72,ForkJoinPool.commonPool-worker-2,5,VirtualThreads]
 : Total time: 210 ms
```

Blocking code + Virtual Threads

Results between 50 ms and 234ms


```
 : Thread before DB query - VirtualThread[#67,tomcat-handler-3]/runnable@ForkJoinPool-1-worker-1
_0 on c1_0.id=u1_0.city_id where u1_0.name=?
 : Thread before calling weather API and after DB query - VirtualThread[#67,tomcat-handler-3]/runnable@ForkJoinPool-1-worker-1
 : Thread after receiving weather API response - VirtualThread[#67,tomcat-handler-3]/runnable@ForkJoinPool-1-worker-1
 : Time of execution: 0.053 ms
```

Completable Future <-> Virtual Threads

Both solutions, in different configurations, execute the same code repeated n times.

Task

```
Runnable action = () -> {
    try {
        Thread.sleep( millis: 100);
    } catch (InterruptedException e) {
        throw new RuntimeException(e);
    }
};
```

Virtual Threads

10 000 -> `Virtual Thread took 1,360 seconds`

50 000 -> `Virtual Thread took 2,084 seconds`

100 000 -> `Virtual Thread took 2,386 seconds`

200 000 -> `Virtual Thread took 2,868 seconds`

500 000 -> `Virtual Thread took 3,034 seconds`

1 000 000 -> `Virtual Thread took 3,691 seconds`

Example of solution

```java
public static void main(String[] args)  {

    Runnable action = () -> {
        try {
            Thread.sleep( millis: 100);
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
    };

    long start = System.currentTimeMillis();
    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
        for (int i = 0; i < 1_000_000; i++) {
            executor.submit(action);
        }
    }

    long time = System.currentTimeMillis() - start;
    System.out.printf("%s took %.3f seconds%n", "Virtual Thread", time / 1000.0);
}
```

Completable Future + Virtual Executor

10 000 -> `CompletableFuture took 1,559 seconds`

50 000 -> `CompletableFuture took 1,792 seconds`

100 000 -> `CompletableFuture took 2,010 seconds`

200 000 -> `CompletableFuture took 2,184 seconds`

500 000 -> `CompletableFuture took 2,599 seconds`

1 000 000 -> `CompletableFuture took 3,820 seconds`

```java
public static void main(String[] args) {

    Runnable action = () -> {
        try {
            Thread.sleep( millis: 100);
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
    };

    long start = System.currentTimeMillis();

    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
        CompletableFuture<?>[] futures = IntStream.range(0, 1_000_000) IntStream
                .mapToObj(_ -> CompletableFuture.runAsync(action)) Stream<CompletableFuture<...>>
                .toArray(CompletableFuture[]::new);

        CompletableFuture.allOf(futures).join();

    }

    long time = System.currentTimeMillis() - start;
    System.out.printf("%s took %.3f seconds%n", "CompletableFuture", time / 1000.0);
}
```

Completable Future + Platform Executor n = 1000

10 000 -> `CompletableFuture took 1,297 seconds`

50 000 -> `CompletableFuture took 5,707 seconds`

100 000 -> `CompletableFuture took 11,196 seconds`

200 000 -> `CompletableFuture took 22,195 seconds`

500 000 -> `CompletableFuture took 55,319 seconds`

1 000 000 -> `CompletableFuture took 110,236 seconds`

```java
public static void main(String[] args) {

    Runnable action = () -> {
        try {
            Thread.sleep( millis: 100);
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
    };

    long start = System.currentTimeMillis();

    try (var executor = Executors.newFixedThreadPool( nThreads: 1000)) {
        CompletableFuture<?>[] futures = IntStream.range(0, 10_000) IntStream
                .mapToObj(_ -> CompletableFuture.runAsync(action, executor)) Stream<CompletableFuture<...>>
                .toArray(CompletableFuture[]::new);

        CompletableFuture.allOf(futures).join();

    }
    long time = System.currentTimeMillis() - start;
    System.out.printf("%s took %.3f seconds%n", "CompletableFuture", time / 1000.0);
}
```

Completable Future + default ForkJoinPool

10 000

```
CompletableFuture took 156,523 seconds
```

```java
public static void main(String[] args) {

    Runnable action = () -> {
        try {
            Thread.sleep( millis: 100);
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
    };

    long start = System.currentTimeMillis();

    CompletableFuture<?>[] futures = IntStream.range(0, 10_000) IntStream
            .mapToObj(_ -> CompletableFuture.runAsync(action)) Stream<CompletableFuture<...>>
            .toArray(CompletableFuture[]::new);

    CompletableFuture.allOf(futures).join();


    long time = System.currentTimeMillis() - start;
    System.out.printf("%s took %.3f seconds%n", "CompletableFuture", time / 1000.0);
}
```

Summary

On the Virtual Thread executor, both `submit()` and `runAsync()` methods showed comparable execution times. In contrast, running `runAsync()` on a platform thread executor with a pool size of 1000 was significantly slower.

This highlights that Virtual Threads provide much better performance and scalability for concurrent tasks compared to traditional Platform Threads.

Using `ExecutorService.submit()` with Virtual Threads:

- simplifies code by avoiding complex callback management
- reduces overhead from extra abstractions,
- offers clearer control over task execution and synchronization.

Tests show that for large scale blocking tasks, direct use of `Virtual Threads` is often faster and simpler, making it a compelling choice for modern concurrent programming.

The Virtual Way won 🔥

Spring Final Test (to ignore)

Running `IntStream.range(0, 100000)` with virtual threads caused minor database issues from too many connections. Virtual threads weren't enabled by just adding properties—I had to configure the executor explicitly.

```
.mapToObj(_ -> CompletableFuture.runAsync(action, virtualThreadExecutor))
```

Spring Final Test (to ignore)

```
runScenarioParallelBlockingCode(() ->
        userService.getUserWithWeatherVirtual( name: "Konrad Kowalczyk")
);
```

```
runScenarioParallel(() ->
        userService.getUserWithWeather( name: "Konrad Kowalczyk")
);
```

Without Virtual Threads

```
Blocking code took 138,218 seconds
```

```
CompletableFuture took 53,249 seconds
```

With Virtual Threads

~~Blocking code took 160,970 seconds~~

~~CompletableFuture took 45,450 seconds~~

~~CompletableFuture took 51,759 seconds~~

Questions?

Thank you for your attention!

sources:

- https://openjdk.org/
- Presentation “Virtual Threads, 2 years later” by Adam Warski at the Devoxx conference
- Presentation “Continuations: The magic behind virtual threads in Java” by Balkrishna Rawool at the Devoxx conference